CN-Stats-Lec06-Probability & Probability Distribution in Statistics

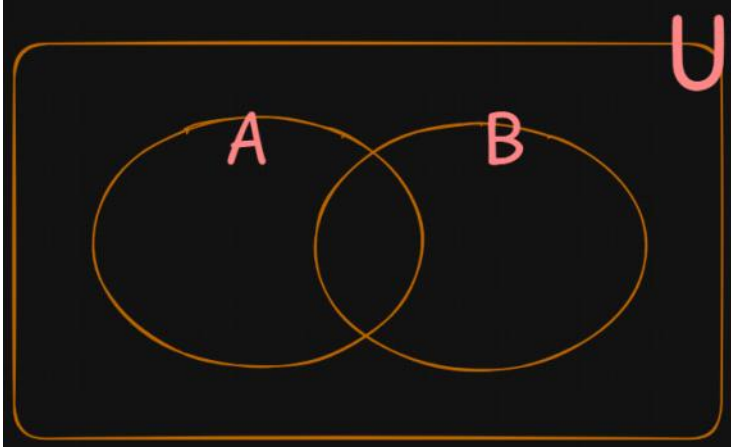
Probability & Probability Distribution in Statistics

🎯 Session Objectives:

- ✅ Use the Law of Total Probability and Bayes' Theorem to update beliefs with new evidence.
- ✅ Discrete (PMF): compute simple probabilities.
- ✅ Continuous: plot & read histograms with Matplotlib.
- ✅ Distributions: Normal (μ,σ) | Standard Normal (0,1) | Poisson (λ).
- ✅ Simulation: show sample mean → true mean as n grows.

Essential Probability Formulas

Union (General) $P(A \cup B) = P(A) + P(B) - P(A \cap B)$	**Union (Mutually Exclusive)** $P(A \cup B) = P(A) + P(B)$
Intersection (General) $P(A \cap B) = P(A) \times P(B \mid A)$	**Intersection (Independent)** $P(A \cap B) = P(A) \times P(B)$
Difference $P(A - B) = P(A) - P(A \cap B)$	**Complement** $P(\overline{A} \cap \overline{B}) = 1 - P(A \cup B)$



$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

```python
# Importing all Dependencies
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```python
# Mutually Exclusive Events
# Rolling a 3 or Rolling a 5
trails = 10000
die_roll = [random.randint(1,6) for _ in range(trails)]
print(die_roll)
```

```
[2, 2, 5, 6, 2, 4, 3, 2, 2, 4, 6, 3, 3, 4, 1, 6, 6, 5, 5, 6, 2, 1, 1, 6, 3, 1, 4, 2, 3, 6, 2, 1, 2, 6, 6, 2,
1, 3, 6, 3, 4, 6, 2, 2, 6, 3, 4, 4, 4, 1, 6, 6, 2, 4, 6, 2, 1, 4, 4, 6, 1, 2, 6, 5, 1, 2, 2, 5, 2, 5, 1, 3,
6, 1, 2, 5, 5, 3, 4, 3, 3, 3, 1, 5, 2, 2, 1, 3, 6, 6, 2, 6, 5, 3, 3, 2, 3, 2, 1, 6, 5, 2, 1, 4, 4, 3, 6, 4,
3, 2, 5, 4, 2, 2, 4, 1, 1, 5, 1, 2, 4, 5, 3, 4, 2, 1, 6, 1, 1, 6, 1, 2, 6, 1, 6, 2, 4, 3, 5, 5, 5, 5, 5, 6,
3, 6, 4, 5, 2, 2, 5, 2, 2, 4, 1, 1, 1, 5, 2, 4, 4, 5, 2, 1, 6, 4, 6, 2, 6, 2, 1, 4, 5, 4, 6, 5, 4, 6, 5, 5,
2, 2, 2, 4, 1, 6, 3, 3, 3, 3, 1, 5, 6, 1, 3, 4, 3, 3, 5, 2, 1, 3, 4, 1, 1, 4, 5, 6, 5, 4, 6, 1, 4, 4, 6, 5,
3, 5, 4, 4, 1, 3, 2, 4, 3, 5, 2, 3, 6, 5, 3, 2, 6, 5, 2, 3, 3, 1, 2, 5, 4, 1, 2, 1, 4, 6, 5, 2, 3, 1, 5, 3,
1, 1, 5, 6, 5, 3, 1, 1, 2, 6, 3, 1, 2, 1, 5, 5, 4, 3, 6, 6, 4, 1, 2, 5, 2, 5, 1, 2, 6, 2, 3, 2, 1, 6, 4, 3,
1, 4, 6, 4, 6, 3, 1, 3, 4, 6, 2, 1, 6, 1, 1, 3, 3, 3, 3, 2, 5, 5, 1, 5, 2, 1, 5, 4, 1, 6, 5, 2, 6, 3, 3, 2,
2, 6, 1, 3, 4, 2, 4, 4, 5, 5, 2, 3, 1, 5, 1, 5, 1, 5, 2, 1, 3, 6, 2, 3, 2, 5, 2, 3, 3, 6, 4, 2, 6, 5, 6, 3,
4, 1, 6, 6, 3, 1, 5, 2, 6, 3, 1, 6, 5, 3, 5, 3, 1, 6, 5, 4, 1, 6, 1, 5, 6, 6, 4, 1, 6, 3, 2, 3, 6, 4, 6, 3,
5, 6, 3, 2, 1, 5, 1, 1, 1, 1, 4, 3, 2, 4, 5, 4, 1, 3, 6, 3, 2, 4, 5, 3, 1, 5, 4, 6, 3, 6, 5, 1, 2, 6, 1, 5,
4, 1, 1, 1, 2, 2, 4, 1, 1, 5, 2, 3, 6, 6, 5, 3, 4, 5, 6, 1, 2, 5, 3, 6, 5, 4, 6, 4, 1, 5, 1, 1, 1, 3, 2, 6,
2, 3, 6, 5, 6, 2, 1, 4, 5, 4, 5, 2, 2, 2, 2, 3, 3, 5, 5, 3, 1, 3, 1, 5, 2, 5, 5, 5, 3, 4, 5, 5, 3, 3, 4, 1,
4, 3, 6, 2, 5, 5, 4, 4, 2, 1, 2, 6, 2, 6, 5, 5, 3, 6, 3, 1, 3, 1, 3, 5, 3, 6, 2, 2, 4, 3, 2, 1, 5, 4, 6, 3,
3, 2, 4, 4, 3, 6, 3, 1, 2, 5, 2, 5, 6, 6, 5, 4, 6, 2, 2, 4, 3, 4, 3, 3, 3, 1, 5, 5, 3, 4, 6, 6, 1, 4, 4, 4,
6, 6, 5, 2, 3, 1, 6, 1, 3, 5, 2, 3, 6, 4, 5, 2, 5, 3, 6, 5, 6, 3, 4, 3, 3, 1, 1, 2, 6, 1, 1, 2, 4, 6, 4, 4,
```

```python
exclusive_events = sum(True for die in die_roll if die == 3 or die == 5)
exclusive_events
```

```
3284
```

```python
probability_events = exclusive_events / trails
probability_events
```

```
0.3284
```

```python
1/3
```

```
0.3333333333333333
```

```python
# Conditional Probability: Card Example
# Drawing two cards from a deck "without replacement" demonstrates conditional probability perfectly.
from itertools import product
deck = list(product(['1','2','3','4','5','6','7','8','9','10','J','Q','K'],
                    ['Heart','Diamond','Spade','Club']))
deck
```

```
[('1', 'Heart'),
 ('1', 'Diamond'),
 ('1', 'Spade'),
 ('1', 'Club'),
 ('2', 'Heart'),
 ('2', 'Diamond'),
 ('2', 'Spade'),
 ('2', 'Club'),
 ('3', 'Heart'),
 ('3', 'Diamond'),
 ('3', 'Spade'),
 ('3', 'Club'),
 ('4', 'Heart'),
 ('4', 'Diamond'),
 ('4', 'Spade'),
 ('4', 'Club'),
 ('5', 'Heart'),
```

```python
random.shuffle(deck)
deck
```

```
[('Q', 'Diamond'),
 ('Q', 'Spade'),
 ('5', 'Diamond'),
 ('8', 'Club'),
 ('5', 'Spade'),
 ('2', 'Heart'),
 ('5', 'Heart'),
 ('4', 'Heart'),
 ('6', 'Diamond'),
 ('J', 'Club'),
 ('9', 'Heart'),
 ('3', 'Diamond'),
 ('1', 'Spade'),
 ('6', 'Club'),
 ('J', 'Diamond'),
 ('9', 'Diamond'),
 ('4', 'Club'),
```

```python
# Conditional Probability : P(A|B) = P(A ∩ B) / P(B) where P(B) !=0
# Find the Probability of Pick an King given that 'Ace' has already drawn.
ace_draws = 0
ace_king_draws = 0
trails = 10000

for _ in range(trails):
    random.shuffle(deck)
    first_card = deck.pop() # 52 cards
    second_card = deck.pop() # 51 cards
    if first_card[0] == '1':
        ace_draws +=1
        if second_card[0] == 'K':
            ace_king_draws +=1
    deck.append(first_card)
    deck.append(second_card)
# Shorthand If-Else : value1 if condition1 else value2
probability_events = (ace_king_draws / ace_draws if ace_draws != 0 else 0)
probability_events
```

```
0.0890840652446675
```

```python
4/51
```

```
0.0784313725490196
```

```python
first_card
```

```
('9', 'Diamond')
```

```python
second_card
```

```
('Q', 'Club')
```

```python
# Rolling a Dice -> Conditional Probability
# Conditional Probability : P(A|B) = P(A ∩ B) / P(B) where P(B) !=0
# rolling a dice > 3 : [1,2,3,4,5,6] -> [4,5,6] -> 3/6 -> 1/2 -> 50%
# Given that dice > 3 : Picking the even value from the remaining set. -> [4,6] -> 2/6 -> 33.33%
dice_roll = [random.randint(1,6) for _ in range(10000)]
dice_above_3 = 0
even_dice_above_3 = 0
trails = 10000

for dice in dice_roll:
    if dice > 3:
        dice_above_3 +=1
        if dice % 2 == 0:
            even_dice_above_3 +=1

conditional_events = (even_dice_above_3 / dice_above_3 if dice_above_3 != 0 else 0)
conditional_events
```

```
0.6606253734315873
```

```python
# Baye's Therorm
import random
Probability_A = 0.01 # P(A)
Probability_B_given_A = 0.99 # P(B|A)
Probability_B_given_not_A = 0.05 # P(B|~A)
Probability_not_A = 1 - Probability_A # 0.99 # P(~A)

trails = 10000
count_A_and_B = 0 # has_disease & test_positive
count_B = 0 # no_disease & test_positive

for _ in range(trails):
    has_disease = random.random() < Probability_A # True/False
    if has_disease:
        test_positivity = random.random() < Probability_B_given_A # True/False
    else:
        test_positivity = random.random() < Probability_B_given_not_A # True/False

    if has_disease and test_positivity:
        count_A_and_B +=1
    if test_positivity:
        count_B +=1

Probability_of_A_given_B = (count_A_and_B / count_B if count_B !=0 else 0)
Probability_of_A_given_B
```

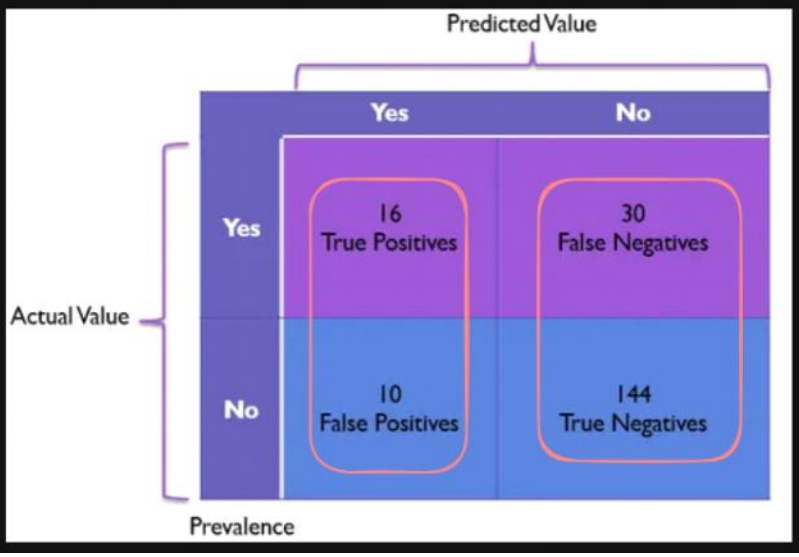
```
0.15639810426540285
```

```python
random.random()
```

```
0.08240264626486948
```

```
Only 15.6% probability of having the disease despite a positive test!
```

Confusion Matrix



```python
import pandas as pd

# Sample data creation
data = {
'Default': ['Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'No'],
'CreditScore': ['Poor', 'Good', 'Poor', 'Average', 'Poor',
'Good', 'Poor', 'Average', 'Poor', 'Good'],
'PastDefault': ['Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes', 'No', 'No']
}

df = pd.DataFrame(data)

# Step 1: Calculate P(Default | Poor CreditScore)
# P(Default | Poor) = P(Default and Poor) / P(Poor)

# Calculate P(Default and Poor)
default_and_poor = df[(df['Default'] == 'Yes') & (df['CreditScore'] == 'Poor')].shape[0]

# Calculate P(Poor)
poor_credit_score = df[df['CreditScore'] == 'Poor'].shape[0]

# Total number of customers
total_customers = df.shape[0]

# Calculate the probabilities
P_default_and_poor = default_and_poor / total_customers
P_poor_credit_score = poor_credit_score / total_customers

# Conditional probability P(Default | Poor)
P_default_given_poor = P_default_and_poor / P_poor_credit_score

print(f"Probability of Default given Poor Credit Score: {P_default_given_poor:.2f}")

# Step 2: Use Bayes' Theorem to update the probability
# We want to calculate P(Default | Poor CreditScore and Past Default)
# P(A | B) = [P(B | A) * P(A)] / P(B)

# P(Past Default | Default)
past_default_given_default = df[(df['Default'] == 'Yes') & (df['PastDefault'] == 'Yes')].shape[0] / default_and_poor

# P(Default)
P_default = default_and_poor / total_customers

# P(Poor CreditScore and Past Default)
poor_and_past_default = df[(df['CreditScore'] == 'Poor') & (df['PastDefault'] == 'Yes')].shape[0]
P_poor_and_past_default = poor_and_past_default / total_customers

# Calculate P(Default | Poor CreditScore and Past Default) using Bayes' Theorem
P_default_given_poor_and_past_default = (past_default_given_default * P_default) / P_poor_and_past_default

print(f"Updated Probability of Default given Poor Credit Score and Past Default:
        {P_default_given_poor_and_past_default:.2f}")
```

```python
import pandas as pd

# Sample data creation
data = {
'Default': ['Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'No'],
'CreditScore': ['Poor', 'Good', 'Poor', 'Average', 'Poor',
'Good', 'Poor', 'Average', 'Poor', 'Good'],
'PastDefault': ['Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes', 'No', 'No']
}

df = pd.DataFrame(data)
df
```

	Default	CreditScore	PastDefault
0	Yes	Poor	Yes
1	No	Good	No
2	Yes	Poor	Yes
3	No	Average	No
4	Yes	Poor	Yes
5	No	Good	No
6	Yes	Poor	Yes
7	No	Average	Yes
8	Yes	Poor	No
9	No	Good	No

```python
# Step 1: Calculate P(Default | Poor CreditScore)
# P(Default | Poor) = P(Default and Poor) / P(Poor)

# Calculate P(Default and Poor)
default_and_poor = df[(df['Default'] == 'Yes') & (df['CreditScore'] == 'Poor')]
default_and_poor
```

	Default	CreditScore	PastDefault
0	Yes	Poor	Yes
2	Yes	Poor	Yes
4	Yes	Poor	Yes
6	Yes	Poor	Yes
8	Yes	Poor	No

```python
default_and_poor.shape
```

```
(5, 3)
```

```python
default_and_poor.shape[0]
```

```
5
```

```python
# Step 1: Calculate P(Default | Poor CreditScore)
# P(Default | Poor CreditScore) = P(Default and Poor CreditScore) / P(Poor CreditScore)

# Calculate P(Default and Poor CreditScore)
default_and_poor = df[(df['Default'] == 'Yes') & (df['CreditScore'] == 'Poor')].shape[0]
default_and_poor
```

```
5
```

```python
# Calculate P(Poor CreditScore)
poor_credit_score = df[df['CreditScore'] == 'Poor']
poor_credit_score
```

	Default	CreditScore	PastDefault
0	Yes	Poor	Yes
2	Yes	Poor	Yes
4	Yes	Poor	Yes
6	Yes	Poor	Yes
8	Yes	Poor	No

```python
poor_credit_score.shape[0]
```

```
5
```

```python
poor_credit_score = df[df['CreditScore'] == 'Poor'].shape[0]
poor_credit_score
```

```
5
```

```python
# Total number of customers
total_customers = df.shape[0] # 10

# Calculate the probabilities
P_default_and_poor = default_and_poor / total_customers # 5/10 -> 1/2 -> 0.5
P_poor_credit_score = poor_credit_score / total_customers # 5/10 -> 1/2 -> 0.5
print(P_default_and_poor)
print(P_poor_credit_score)
```

```
0.5
0.5
```

```python
# Conditional probability P(Default | Poor)
P_default_given_poor = P_default_and_poor / P_poor_credit_score

print(f"Probability of Default given Poor Credit Score: {P_default_given_poor:.2f}")
```

```
Probability of Default given Poor Credit Score: 1.00
```

```python
# Step 2: Use Bayes' Theorem to update the probability
# We want to calculate P(Default | Poor CreditScore and Past Default)
# P(A | B) = [P(B | A) * P(A)] / P(B)

# P(Past Default | Default)
past_default_given_default = df[(df['Default'] == 'Yes') & (df['PastDefault'] == 'Yes')]
past_default_given_default
```

	Default	CreditScore	PastDefault
0	Yes	Poor	Yes
2	Yes	Poor	Yes
4	Yes	Poor	Yes
6	Yes	Poor	Yes

```python
past_default_given_default = (
    df[(df['Default'] == 'Yes') & (df['PastDefault'] == 'Yes')].shape[0] / default_and_poor
)
past_default_given_default
```

```
0.8
```

```python
# P(Default)
P_default = default_and_poor / total_customers
P_default
```

```
0.5
```

```python
# P(Poor CreditScore and Past Default)
poor_and_past_default = df[(df['CreditScore'] == 'Poor') & (df['PastDefault'] == 'Yes')]
poor_and_past_default
```

	Default	CreditScore	PastDefault
0	Yes	Poor	Yes
2	Yes	Poor	Yes
4	Yes	Poor	Yes
6	Yes	Poor	Yes

```python
poor_and_past_default = df[(df['CreditScore'] == 'Poor') & (df['PastDefault'] == 'Yes')].shape[0]
poor_and_past_default
```

```
4
```

```python
P_poor_and_past_default = poor_and_past_default / total_customers
P_poor_and_past_default
```

```
0.4
```

```python
# Calculate P(Default | Poor CreditScore and Past Default) using Bayes' Theorem
P_default_given_poor_and_past_default = (past_default_given_default * P_default) / P_poor_and_past_default

print(f"Updated Probability of Default given Poor Credit Score and Past Default: {P_default_given_poor_and_pas
```

```
Updated Probability of Default given Poor Credit Score and Past Default: 1.00
```

Imagine you are a data analyst working for a telecom company.
Your task is to analyze customer behavior and predict whether a customer is likely to churn (i.e., leave the service). You have a dataset that includes various features such as customer demographics, service usage patterns, and historical data on whether customers have churned in the past.

You want to calculate the probability that a customer will churn given that they have high monthly charges.

Total number of customers: 5,000
Number of customers with high monthly charges: 1,500
Number of customers who churned: 800
Number of customers who churned and had high monthly charges: 300

```python
# Conditional Probability
# P(Churn | High Charges) = P(Churn + High Charges) / P(High Charges)
# Given Information
total_customers = 5000
high_monthly_charges_customers =  1500
churned_customers = 800
high_monthly_charges_churned_customers = 300

# P(Churn + High Charges)
P_Churn_given_High_Charges = high_monthly_charges_churned_customers / total_customers
P_Churn_given_High_Charges
```

```
0.06
```

```python
P_High_Charges = high_monthly_charges_customers / total_customers
P_High_Charges
```

```
0.3
```

```python
# P(Churn | High Charges) = P(Churn + High Charges) / P(High Charges)
Conditional_Prob = P_Churn_given_High_Charges / P_High_Charges
Conditional_Prob
```

```
0.2
```

Combination Formula

$$^{n}C_{r} = \frac{n!}{(n-r)!r!}$$